# Module 9 - Procedural SQL

## Extending SQL with Procedural Logic

# Learning Outcomes – Procedural SQL (PL/pgSQL)

**By the end of this module, you will be able to:**

- Explain the role of procedural SQL in automating business rules within a relational database.

- Create and test **triggers** and **stored procedures** using **PL/pgSQL** in PostgreSQL.

# Use Case: Automating Product Reordering

**Scenario – SaleCo Inventory Management System**

- Ensure there are always enough products available for customers.
- Automatically trigger a **vendor reorder flag** when **Quantity on Hand (QoH)** falls below the **minimum stock level**.

How can we automate product reordering using procedural SQL?

# Automating Product Ordering

**Business Procedure:**

1. Ensure **Quantity on Hand (QoH)** values are accurate and consistent.
2. If **QoH < Min_Allowable**, automatically **generate a reorder** for that product.

💡 *Automation ensures stock levels stay consistent without manual intervention.*

# SQL is Declarative Not Procedural

- SQL specifies **what** to do, not **how** to do it.
- SQL alone does **not support conditional logic or looping operations**.
- To automate business rules, we use a **procedural extension** such as **PL/pgSQL**.

💡 *Declarative SQL describes the goal; procedural SQL defines the process.*

# SQL is Declarative Not Procedural

- SQL cannot express *how* to perform operations — only *what* to do.
- As a result, **business rules must be duplicated across multiple programs.**
- This leads to **inconsistent logic** and **high maintenance overhead.**

💡 *Procedural SQL (PL/pgSQL) allows us to centralize this logic inside the database.*

# A Better Approach – Centralizing Business Logic

- **Centralize critical code** so all applications call the same shared logic.
- Especially effective in **distributed** and **object-oriented** systems.
- Modern RDBMSs extend SQL with procedural features such as:
  - **Flow control:** `IF–THEN–ELSE`, `LOOP`, `EXIT WHEN`
  - **Variable declaration & assignment**
  - **Error handling and exceptions**

💡 *PostgreSQL implements these features through its procedural language, PL/pgSQL.*

# Procedural SQL

# Persistent Stored Modules (PSM)

- A **Persistent Stored Module (PSM)** is a block of SQL and procedural code **stored and executed on the DBMS server**.
- Encapsulates **business logic** that can be **shared and reused** across applications.
- The **SQL:1999 (SQL/PSM)** standard defines this concept.
- In **PostgreSQL**, stored functions and procedures written in **PL/pgSQL** are examples of PSMs.

💬 *PSMs centralize logic inside the database — improving consistency and reducing duplication.*

# Writing a PL/pgSQL Code Block

- Use `BEGIN...END` to group SQL and procedural statements into one executable block.
- Use `RAISE NOTICE` to display messages and debug your block.
- Use `RAISE EXCEPTION` to handle or test error conditions.
- PL/pgSQL blocks form the basis for **triggers** and **stored procedures**.

💡 *You can run standalone PL/pgSQL blocks using `DO $$ ... $$;` in pgAdmin.*

```
DO $$
BEGIN
  RAISE NOTICE 'Hello from PL/pgSQL!';
END;
$$;
```

# Declaring and Using Variables in PL/pgSQL

- Define variables in the **DECLARE** section before BEGIN.
- Each variable has a **name**, **data type**, and optional **initial value**.
- Use **:=** to assign values within the block.
- Use **SELECT ... INTO** to assign query results.

```
DO $$
DECLARE
  discount numeric := 0.10;
  price    numeric := 100;
  total    numeric;
BEGIN
  SELECT price * (1 - discount) INTO total;
  RAISE NOTICE 'Discounted price = %', total;
END;
$$;
```

# While Loops in PL/pgSQL

A **WHILE loop** repeats a statement block **while a condition is true.**

Syntax:

```
WHILE condition LOOP
  -- statements
END LOOP;
```

Example:

```
counter := 1;
WHILE counter <= 3 LOOP
  RAISE NOTICE 'Counter = %', counter;
  counter := counter + 1;
END LOOP;
```

# FOR Loops in PL/pgSQL

A **FOR loop** repeats a block of statements for each value in a range or query result.

Syntax:

```
FOR variable IN lower..upper LOOP
  -- statements
END LOOP;
```

Example:

```
FOR i IN 1..5 LOOP
  RAISE NOTICE 'Iteration %', i;
END LOOP;
```

# Conditional Logic in PL/pgSQL

Use `IF`, `ELSIF`, and `ELSE` to control flow in your code.

Syntax:

```
IF condition THEN
  -- statements
ELSIF other_condition THEN
  -- statements
ELSE
  -- statements
END IF;
```

Example:

```
IF score >= 90 THEN
  RAISE NOTICE 'Grade: A';
ELSIF score >= 80 THEN
  RAISE NOTICE 'Grade: B';
ELSE
  RAISE NOTICE 'Grade: C or below';
END IF;
```

# CASE Statements in PL/pgSQL

Use CASE to handle multiple possible values of an expression.

Syntax:

```
CASE expression
  WHEN value1 THEN
    -- statements
  WHEN value2 THEN
    -- statements
  ELSE
    -- default statements
END CASE;
```

Example:

```
CASE status
  WHEN 'new' THEN RAISE NOTICE 'Order received.';
  WHEN 'shipped' THEN RAISE NOTICE 'Order has been shipped.';
  ELSE RAISE NOTICE 'Unknown status.';
END CASE;
```

# Procedural Language / SQL (PL/pgSQL)

- **PL/pgSQL** merges SQL with programming constructs (variables, loops, conditionals).
- Procedural logic can be **stored inside the database**, not just in applications.
- Types of PL/pgSQL code:
  - **Anonymous blocks** (`DO $$ ... $$;`)
  - **Triggers** (automatic actions)
  - **Stored procedures** (`CREATE PROCEDURE`)
  - **Functions** (`CREATE FUNCTION`)
- Can be executed **directly by users** or **indirectly through applications**.

💬 *PL/pgSQL brings programming logic into the database layer for efficiency and consistency.*

# Triggers – Automating Business Rules in the Database

Example: Automatically Reordering Products When Stock Runs Low

Triggers let the database react automatically to events like INSERT, UPDATE, or DELETE.

# Trigger Syntax in PostgreSQL

```
CREATE [ OR REPLACE ] TRIGGER trigger_name
{ BEFORE | AFTER | INSTEAD OF } { INSERT | UPDATE | DELETE | TRUNCATE }
ON table_name
[ FOR EACH ROW | FOR EACH STATEMENT ]
EXECUTE FUNCTION function_name();
```

- Trigger function defines the logic and must return `trigger`.
- `BEFORE` triggers validate or modify data.
- `AFTER` triggers perform follow-up actions.
- Always call a **stored function** written in PL/pgSQL.

💬 *PostgreSQL separates the trigger definition from the trigger logic itself.*

# Setting Product Reorder Flag - Version 1

**Goal:** Automatically mark products for reorder when QOH ≤ MIN_STOCK.

**Trigger Function:**

```
CREATE OR REPLACE FUNCTION product_reorder_trg_v1()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  UPDATE product
     SET reorder = TRUE
   WHERE qoh <= min_stock;
  RETURN NULL;
END;
$$;
```

**Trigger Definition:**

```
CREATE TRIGGER trg_product_reorder_v1
AFTER INSERT OR UPDATE OF qoh
ON product
FOR EACH STATEMENT
EXECUTE FUNCTION product_reorder_trg_v1();
```

# Components of a Trigger

Trigger **Timing**

Triggering **Events**

```
CREATE TRIGGER trg_product_reorder_v1
AFTER INSERT OR UPDATE OF qoh
ON product
FOR EACH STATEMENT
EXECUTE FUNCTION product_reorder_trg_v1();
```

Trigger **Level**

Trigger **Function**

# Handling Multiple Triggering Conditions

- Changing **min_stock** should also update the **reorder** flag.
- The previous trigger didn't respond because only `UPDATE OF qoh` was included.
- ✅ Update the trigger to listen for **multiple columns**:

```
CREATE TRIGGER trg_product_reorder_v2
AFTER INSERT OR UPDATE OF qoh, min_stock
ON product
FOR EACH STATEMENT
EXECUTE FUNCTION product_reorder_trg_v2();
```

💬 *Now the trigger fires when either quantity or minimum stock level changes.*

# Updating the Trigger Logic with CASE Expressions

- Problem: trigger sets `reorder` to TRUE but never resets it.
- Solution: use a **CASE expression** to handle both conditions.

Notice the use of **CASE** to compute P_REORDER

```
-- Trigger function
CREATE OR REPLACE FUNCTION product_reorder_trg_v3()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  UPDATE product
     SET reorder = CASE
                      WHEN qoh <= min_stock THEN TRUE
                      ELSE FALSE
                   END;
  RETURN NULL;
END;
$$;
```

# Target Only Affected Rows

**Problem:** Our current trigger updates **all** rows in `product`, not just the ones modified.

**Solution:**

Use `FOR EACH ROW` and

set the flag directly on `NEW`.

```sql
-- Trigger (fires for qoh or min_stock changes)
CREATE TRIGGER trg_product_reorder_v4_row
BEFORE INSERT OR UPDATE OF qoh, min_stock
ON product
FOR EACH ROW
EXECUTE FUNCTION product_reorder_trg_v4_row();
```

# The NEW and OLD Records in Triggers

PostgreSQL provides two pseudo-records inside row-level triggers:

- NEW → the new row after the change
- OLD → the original row before the change
- Used to read or modify values in trigger logic.

```
-- Trigger function
CREATE OR REPLACE FUNCTION product_reorder_trg_v4_row()
RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  NEW.reorder := (NEW.qoh <= NEW.min_stock);
  RETURN NEW;  -- row-level trigger must return NEW/OLD
END;
$$;
```

# Changing Trigger Timing: AFTER → BEFORE

- **AFTER trigger:** runs after the data change — can't modify the new row.
- **BEFORE trigger:** runs before the change — can adjust the row's values directly.
- Switching to `BEFORE` lets us set the `reorder` flag inside `NEW` without another `UPDATE`.

💡 *Use BEFORE triggers when you want to modify or validate data before it's stored.*

# What Is a Trigger?

- A **trigger** is a procedural SQL block that runs **automatically** when a specified **data manipulation event** occurs.
- It is **event-driven** — fired by `INSERT`, `UPDATE`, or `DELETE`.
- Used to **enforce business rules**, **maintain data integrity**, or **audit changes** automatically.

💬 *Triggers make the database proactive — responding to data changes in real time.*

# Notes on Triggers

- Each **trigger** is linked to a **specific table or view**.
- A table can have **multiple triggers** for different events or timings.
- Trigger actions execute **within the same transaction** as the triggering event.
- Triggers are vital for **automating rules** and **maintaining consistency**.
- They **extend the database's processing power** by embedding logic directly inside the DBMS.

💬 *Triggers transform a database from a passive data store into an active, intelligent system.*

# Automating Product Ordering

**The Business Procedure:**

1. Ensure QOH values are up-to-date & consistent
2. If QOH < min_allowable, product must be reordered

# Updating an Attribute in Another Table

**Business Rule:**

When a sale occurs, decrease the **Quantity on Hand (QOH)** for the sold product.

**Implementation:**

Trigger on the **LINE** table updates the **PRODUCT** table automatically.

# Updating Product QOH Value

```
-- Trigger function: reduce product QOH after each sale
CREATE OR REPLACE FUNCTION reduce_product_qoh()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  UPDATE product
     SET qoh = qoh - NEW.qty
   WHERE code = NEW.product_code;
  RETURN NULL;  -- AFTER trigger, no row modification in LINE
END;
$$;

-- Trigger definition on LINE table
CREATE TRIGGER trg_reduce_product_qoh
AFTER INSERT
ON line
FOR EACH ROW
EXECUTE FUNCTION reduce_product_qoh();
```

TRIGGER on LINE table updates PRODUCT table

# Use of Variables Within a Trigger

We also need to update CUS_BALANCE after inserting every new LINE

→ create a trigger for the LINE table that updates a row in the CUSTOMER table. This time using **variables** in our trigger function..

```
-- Trigger definition on LINE
CREATE TRIGGER trg_add_to_customer_balance
AFTER INSERT ON line
FOR EACH ROW
EXECUTE FUNCTION add_to_customer_balance();
```

# Updating Customer Balance

```
-- Trigger function: update customer balance when a new line is inserted
CREATE OR REPLACE FUNCTION add_to_customer_balance()
RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
  v_customer_id  integer;
  v_line_total   numeric;
BEGIN
  -- Step 1: Find which customer this line belongs to
  SELECT i.customer_id
    INTO v_customer_id
    FROM invoice i
   WHERE i.id = NEW.invoice_id;

  -- Step 2: Calculate the total for this line
  v_line_total := NEW.price * NEW.qty;

  -- Step 3: Add the amount to the customer's balance
  UPDATE customer
     SET balance = COALESCE(balance, 0) + v_line_total
   WHERE id = v_customer_id;

  RETURN NULL;  -- AFTER trigger (no modification to LINE row)
END;
$$;
```

Need to find the customer and calculate line total in order to update customer balance

# Stored Procedures

# Stored Procedures in PostgreSQL

- A **stored procedure** is a **named PL/pgSQL program** stored inside the database.
- Like triggers, it resides in the DB and can be reused by multiple applications.
- It can **encapsulate a complete business transaction**, such as:
  - Recording a sale
  - Updating a customer's credit limit
  - Adding a new product or account
- Procedures help **centralize logic**, ensuring consistent business rules and improving performance.

💬 *Procedures are called explicitly — unlike triggers, which run automatically.*

# Advantages of Stored Procedures

- ⚡ **Reduce network traffic & improve performance**
  - SQL executes inside the database — no need to transmit statements over the network.
- 🔁 **Reduce code duplication**
  - Create reusable PL/pgSQL modules that can be called by multiple programs.
- 🧩 **Simplify application development**
  - Centralize logic to minimize errors and reduce development and maintenance costs.

💬 *Stored procedures make applications leaner and databases smarter.*

# Stored Procedure Syntax (PostgreSQL)

```
CREATE [ OR REPLACE ] PROCEDURE procedure_name ( [ parameters ] )
LANGUAGE plpgsql
AS $$
BEGIN
  -- procedural SQL statements
END;
$$;

CALL procedure_name ( [ arguments ] );
```

zero or more **Parameters** passed to procedure

# Notes on Stored Procedures

- Defined using `LANGUAGE plpgsql`.
- Executed with `CALL`, not `EXEC`.
- Performs actions but **does not return a value**.

💬 *Use procedures for business operations that modify data or manage transactions.*

# Example – Assign Additional 5% Discount

```
CREATE OR REPLACE PROCEDURE prc_prod_discount()
LANGUAGE plpgsql
AS $$
BEGIN
  UPDATE product
     SET discount = discount + 0.05
   WHERE qoh >= (min_stock * 2);

  RAISE NOTICE '* * Update finished * *';
END;
$$;

CALL prc_prod_discount();
```

💬 *This stored procedure adjusts product discounts automatically when stock levels are high.*

# Passing Parameters to Stored Procedures

Stored procedures can accept **input parameters**, allowing you to:

- Make the procedure **reusable** for different cases.
- **Avoid hardcoding values** (e.g., discount rates, thresholds).
- Pass values directly from **applications** or other SQL code.

# Make discount percentage not a fixed 5% but an input

```
CREATE OR REPLACE PROCEDURE prc_prod_discount(p_extra numeric)
LANGUAGE plpgsql
AS $$
BEGIN
  UPDATE product
     SET discount = discount + p_extra
   WHERE qoh >= (min_stock * 2);

  RAISE NOTICE 'Applied extra discount of %', p_extra;
END;
$$;

-- Call the procedure with a parameter value
CALL prc_prod_discount(0.05);
```

# More Examples

# Example – Add a New Customer

```
CREATE PROCEDURE prc_cus_add(
  p_code integer,
  p_lastname varchar(15),
  p_firstname varchar(15),
  p_initial char(1),
  p_areacode char(3),
  p_phone char(8)
)
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO customer (code, lastname, firstname, initial, areacode, phone)
  VALUES (p_code, p_lastname, p_firstname, p_initial, p_areacode, p_phone);

  RAISE NOTICE 'Customer %, % added.', p_lastname, p_firstname;
END;
$$;

CALL prc_cus_add(501, 'Turner', 'Lisa', 'A', '604', '555-2025');
```

# Example – Add a New Invoice

```
CREATE OR REPLACE PROCEDURE prc_inv_add(
  p_invoice_number integer,
  p_customer_code  integer,
  p_invoice_date   date
)
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO invoice (id, customer_id, invoice_date)
  VALUES (p_invoice_number, p_customer_code, p_invoice_date);

  RAISE NOTICE 'Invoice % added for customer % on %.',
    p_invoice_number, p_customer_code, p_invoice_date;
END;
$$;

CALL prc_inv_add(301, 501, CURRENT_DATE);
```